Interview Preparation Guide

Prachi Kabra
Data Engineer | Databricks, PySpark, AI / ML

Self-introduction, project explanations with problem statements, and follow-up questions with answers.

1	Self-introduction
2	Project write-ups (problem, flow, role, challenges, impact, spoken version)
3	Follow-up questions and answers
4	Checklist before the interview

Items shown as **[confirm ...]** or **[fill in ...]** depend on details only you know. Replace them with what you actually did before the interview. An honest, slightly different answer beats a polished guess.

1. Self-Introduction

Speak this at a relaxed pace, pausing briefly after each paragraph. Do not memorize it word for word; remember the flow: **Tech Mahindra > Azure and Fabric > Databricks > S&P Global > CI/CD, Docker, Kubernetes > M.Tech and AI > what you want next.**

Your introduction (about 2 minutes)

Hello, thank you for giving me this opportunity to introduce myself.

My name is Prachi Kabra, and I'm a Data Engineer with over five years of experience in data engineering and data platform development.

I started my career at Tech Mahindra in Pune, where I worked on SSIS pipelines and later moved to Snowflake and Matillion for telecom revenue data. I also developed a validation framework to identify billing discrepancies, which helped reduce revenue leakage by around 15%.

After that, I worked on Azure and Microsoft Fabric projects, including automated data processing and document migration using OCR and APIs. In the last couple of years, my primary focus has been on Databricks, PySpark, and Delta Lake.

Currently, I'm working at S&P Global Ratings, where I'm leading two projects. The first is Loan Analysis, where we have developed an automated pipeline that collects data from multiple vendor APIs, validates and combines it with internal data, and publishes the final reports to SharePoint. We also implemented fuzzy matching logic, which improved report accuracy by 40%.

The second project is Loss Stats, where I lead a team of three members. We are migrating a legacy MS Access system to Databricks and implementing automated data validation, incremental data loads, and data quality checks.

Apart from development, I also work on deployment and automation using Git and CI/CD pipelines through DevOps. I have also worked on proof-of-concept projects using Docker and Kubernetes.

Along with my job, I'm pursuing an M.Tech in AI and Machine Learning from BITS Pilani. I'm interested in exploring how AI can be used in practical engineering applications. Recently, my team developed an internal AI agent to create and update features and user stories in DevOps, with human review. I also secured second place in a hackathon at the AI Impact Summit.

In addition, I'm a Databricks Certified Data Engineer, and I conduct Databricks and Spark training sessions for my colleagues. In my free time, I also work on personal projects related to automated trading and machine learning.

Overall, my experience has been mainly in building and improving data pipelines, automating processes, and working with modern data engineering technologies.

Thank you.

2. Project Write-ups

Flow for every project: **Problem > What I built > My role > Challenges > Impact.** Each project ends with a spoken version you can say aloud.

1. Loan Analysis (S&P Global Ratings, Project Lead)

Automated pipeline: vendor APIs to a consolidated loan report on SharePoint

Tech stack: Python, PySpark, SQL, Databricks, Delta Lake, RapidFuzz (fuzzy logic), CI/CD, AWS S3

Problem statement

Loan reports needed data from several vendors plus internal databases. Collecting, cleaning and merging it by hand was slow and error-prone. Vendors named and formatted records differently, so matching them reliably was hard, and stakeholders had no alerts when something failed.

How it works

```
Vendor APIs (multiple)
> Extract (Python, scheduled)
> Cleanse each source separately (PySpark)
> Combine with internal databases (SQL / Delta Lake)
> Fuzzy matching for inconsistent records (RapidFuzz)
> Consolidated report
> Publish to SharePoint via API
> Success / failure email to stakeholders and team
```

Key design points

- **Source-wise cleansing:** each vendor's data is cleaned on its own before merging, so one bad source does not corrupt the rest.
- **Delta Lake:** reliable, versioned storage with ACID transactions for cleaned and final data.
- **Scheduling with a checkpoint:** triggers on Friday and Monday. If Friday's report already exists, Monday's run is skipped, avoiding duplicate work and compute.
- **Alerting:** email notifications for success and failure.
- **Automated publishing:** the final report is pushed to SharePoint through an API.
- **CI/CD:** code is version-controlled and released in a repeatable way.

My role

Led the project: design, development and communication with stakeholders.

Challenges and solutions

Challenge	How it was solved
Names and records did not match across vendors and internal data	Applied fuzzy logic (RapidFuzz) for matching, which improved accuracy by 40%
Friday and Monday runs could repeat the same work	Added a checkpoint that skips Monday if Friday's report exists
Failures going unnoticed	Added success and failure email alerts
Different formats from each vendor	Cleansed each source separately before combining

Impact

Report accuracy improved by 40%. Extraction, cleanup, report generation, publishing and alerting are automated end to end.

Spoken version

Loan Analysis is an automated reporting pipeline I lead at S&P Global Ratings. The problem was that our loan reports needed data from several vendors and our internal databases, and doing that by hand was slow and error-prone. Vendors also named records differently, which made matching difficult. I built a pipeline on Databricks using PySpark and Delta Lake. It extracts data from each vendor's API, cleans each source separately, and combines it with our internal data. For inconsistent records, I used fuzzy logic with the RapidFuzz library, and that improved report accuracy by 40%. The final report is published to SharePoint through an API. It runs on a Friday and Monday schedule, and a checkpoint skips Monday's run if Friday's report is already done. Stakeholders get an email for success or failure. A fully manual process now runs end to end on its own.

RapidFuzz quick reference

RapidFuzz is a fast string matching library that gives a similarity score from 0 to 100. Steps to describe: normalize text, score candidates, apply a threshold, send borderline matches to review.

Scorer	Good for
fuzz.ratio	Simple overall similarity
fuzz.partial_ratio	One string is part of the other
fuzz.token_sort_ratio	Same words in a different order (Smith John vs John Smith)
fuzz.token_set_ratio	Extra or missing words
process.extractOne	Best match for a name from a list of candidates

```
from rapidfuzz import process, fuzz

best = process.extractOne(
vendor_name,
internal_names,
scorer=fuzz.token_sort_ratio,
score_cutoff=90
)
```

2. Loss Stats (S&P Global Ratings, Team Lead of 3)

Migration of a legacy MS Access system to Databricks

Tech stack: Data modeling, Python, PySpark, SQL, Delta tables, Databricks, CI/CD, AWS S3

Problem statement

The business ran on a legacy MS Access system. Processing was manual, hard to scale and hard to audit. Users depended on Access tables, views and subroutines, so the migration had to reproduce the same behavior exactly without breaking the downstream CreditPro team.

How it works

```
User uploads SharePoint template
> Template ingestion
> Multi-layer validation
> Pre-staging checks
> Load into 4 staging tables
> Subroutines executed in between (replicated Access logic)
> ETL testing and validation at each layer
> Load into 16 final-layer tables
> Views refreshed for the CreditPro team
```

Key design points

- **Incremental load (ELT):** only new or changed data is processed instead of reloading everything.
- **Layered validation:** checks at several points so bad data is stopped early.
- **Staging then final:** 4 staging tables hold data for checks, and only validated data is promoted to 16 final tables.
- **Replicating the old system:** all tables, views and subroutines from MS Access rebuilt in the lakehouse.
- **Views refresh:** views refreshed after load so CreditPro always sees current data.
- **CI/CD:** controlled, repeatable releases.

My role

Lead a team of three. Own data modeling and progress reporting to management and DevOps; hands-on with design and ETL testing.

Challenges and solutions

Challenge	How it was solved
Matching legacy Access behavior exactly	Replicated every table, view and subroutine, then compared outputs side by side
Bad data entering through templates	Multi-layer validation and pre-staging checks before any load
Trusting data before promotion	ETL testing and validation at each layer before moving staging to final
Coordinating team and stakeholders	Clear task ownership and regular progress reporting to management and DevOps

Impact

Manual MS Access processing replaced by automated template validation, layered loads and view refresh, on a scalable Databricks foundation.

Spoken version

Loss Stats is a migration project where I lead a team of three. The business was running on a legacy MS Access system that was manual and hard to scale, and our job is to move it to Databricks while reproducing all its tables, views and subroutines. Users upload a SharePoint template. It goes through multi-layer validation and pre-staging checks, and then loads into four staging tables. The old subroutines run in between, and the data is promoted into sixteen final tables. Finally, the views are refreshed for the CreditPro team. We use an incremental ELT approach, so we only process what is new. I own the data modeling and the progress reporting to management and DevOps, and we do ETL testing at every layer before data moves forward.

3. Internal AI Agent for DevOps (Human in the Loop)

Agent that creates and updates features and user stories on DevOps

Tech stack: Spark AIR (internal AI tool), Azure DevOps **[confirm any other components]**

Problem statement

Creating features and user stories on DevOps, and keeping them updated as work changed, was repetitive and took a lot of team time.

How it works

```
Input (requirement / update request)
> Agent drafts feature or user story
> Human reviews and approves <-- human in the loop
> Agent creates or updates the item on DevOps
```

Key design points

- The agent drafts and a human approves, so nothing changes in DevOps without sign-off.
- Handles both creating new items and updating existing ones.
- Built on the internal AI tool Spark AIR.

My role

Part of the team that built it. **[fill in your exact contribution]**

Challenges and solutions

Challenge	How it was solved
Trust: people feared an agent editing DevOps on its own	Human approval before every save
Quality of generated stories	Consistent structure, with human review as the final check
Keeping existing items accurate	Agent updates items rather than creating duplicates

Impact

A lot of time saved on repetitive work **[add a real number, e.g. hours per sprint]**.

Spoken version

Our team spent a lot of time creating and updating features and user stories on DevOps. Using our internal AI tool, Spark AIR, we built an agent that drafts and updates them. A human reviews and approves every change before it is saved, so we keep control and quality. It saved the team a lot of time on repetitive work.

4. VoxShield AI (India AI Impact Buildathon, 2nd Place)

Multi-layer audio forensics API for deepfake voice detection

Tech stack: Python API, custom ML/DL model (wav2vec2), acoustic features (jitter, shimmer, pitch), Gemini and HuggingFace models in the ensemble, web UI **[confirm exact stack]**

Problem statement

AI voice cloning is the fastest-growing fraud vector in India's digital payments. Our deck cites 47% of Indian adults hit by AI voice scams, about Rs 805 crore lost in UPI fraud till Nov 2025, and only a 6% recovery rate. Most detection tools give a single yes/no answer (about 85% accuracy cited), which fails against partial AI splicing: real speech with AI-generated segments inserted at critical moments, such as stating an account number or confirming a transfer.

How it works

```
Voice audio in (MP3 / WAV, base64, 5 languages)
> Layer 1 Audio Intelligence: decode, validate, format and duration checks
> Layer 2 Core Detection: custom ML/DL model, chunk probabilities, AI vs human score
> Layer 3 Forensic Analysis: chunk statistics, splice detection, trend and pattern analysis
> Layer 4 Explainability: human-readable findings
> Layer 5 Risk Assessment: composite score, calibrated confidence, risk level
> Output: HUMAN or AI_GENERATED verdict + confidence + forensic timeline
```

Key design points

- **Chunk-level forensics:** every audio segment is scored, not just one overall score.
- **Splice detection:** sharp jumps between adjacent chunks (over 35% change in AI probability) are flagged as splice points, showing WHERE AI audio was inserted.
- **Forensic timeline:** a visual probability map across time, usable as evidence in fraud investigations.
- **Five Indian languages:** Hindi, English, Tamil, Telugu and Malayalam.
- **Ensemble approach:** acoustic features, a deep learning model and other models combined into one verdict **[confirm which four experts]**.
- **Accuracy over latency:** 15 to 30 seconds per analysis, because banking fraud needs reliable answers, not instant guesses.
- **Delivered as an API** with a web UI for upload or recording, batch testing and API docs.

My role

[fill in your role in the four-member team ANALYTICSWITHANAND]

Challenges and solutions

Challenge	How it was solved
Partial AI splicing hidden inside real speech	Chunk-level probabilities with splice flags on sharp transitions
Hindi and English code-switching causing variance	Flag it and adjust confidence
Human voices that sound robotic (call center agents, nervous speakers)	Uniformity analysis separates natural variance from synthetic consistency
AI-generated intro and outro wrapped around a human middle	Pattern detection for this bookend tactic
Very short audio, heavy noise, new TTS models	Flag low-confidence results, use the ensemble, and use a wav2vec2 deep learning model that generalizes better to unseen TTS

Impact

Placed 2nd at the India AI Impact Buildathon (AI Impact Summit, Feb 2026). The deck reports about 78-82% accuracy in noisy conditions and 85-88% on novel TTS models **[confirm how these were measured]**.

Spoken version

VoxShield AI is a multi-layer audio forensics API we built for the India AI Impact Buildathon at the AI Impact Summit, where we placed second. The problem is voice-clone fraud, which is growing fast in India's digital payments, and most detection tools only give a yes or no answer. Real scammers often splice AI-generated segments into a real voice at critical moments, like confirming a transfer. We built a five-layer pipeline. It validates and decodes the audio, scores every chunk with a custom ML model, runs forensic analysis to find the exact splice points, generates human-readable findings, and produces a calibrated risk score. The output is a verdict, a confidence and a forensic timeline showing where the AI segments appear. It supports Hindi, English, Tamil, Telugu and Malayalam, and we chose accuracy over latency, about 15 to 30 seconds per analysis, because fraud investigations need reliable evidence.

5. RegimeTrader Pro (Hobby Project)

Autonomous trading system using Hidden Markov Models and news sentiment

Tech stack: Python, Hidden Markov Models, Alpaca paper trading API, FastAPI, Yahoo Finance, Google News, Reddit

Problem statement

Most trading bots react to price alone and behave the same in every market condition. I wanted a system that first understands the market state and then decides how to trade.

How it works

```
10 years of daily prices, 12 Nasdaq stocks
> HMM detects regime: Bull / Bear / Neutral / Crash / Euphoria
Real-time news (Yahoo Finance, Google News, Reddit)
> Sentiment score
> Decision: trade or not, intraday or positional
> Orders via Alpaca paper trading API
> FastAPI dashboard: positions, P&L, news signals
```

Key design points

- HMM gives a regime, and sentiment confirms or challenges it.
- Decides how to trade (intraday vs holding), not just whether.
- Risk controls: circuit breakers, kill switch, dynamic position sizing.
- Paper trading only, so no real money is at risk.

My role

Designed and built end to end as a personal project.

Challenges and solutions

Challenge	How it was solved
Price data alone gives false signals	Combined with news sentiment
Runaway losses in an autonomous system	Circuit breakers and a kill switch
Risk differs by market state	Dynamic position sizing
Noisy social and news data	Multiple sources combined into one sentiment signal

Impact

Runs autonomously with built-in risk controls and a live monitoring dashboard.

Spoken version

As a hobby project, I built an automated stock trading system called RegimeTrader Pro that uses Hidden Markov Models to detect market regimes: Bull, Bear, Neutral, Crash and Euphoria. It trains on 10 years of daily price data across 12 Nasdaq stocks. The system combines HMM-based regime detection with real-time news sentiment from Yahoo Finance, Google News and Reddit to decide not just whether to trade, but whether to trade intraday or hold positionally. I integrated it with Alpaca's paper trading API for order execution and built a FastAPI dashboard to monitor live positions, P&L and news signals. It runs autonomously with circuit breakers, a kill switch and dynamic position sizing.

6. Data Validation Framework (AI + Great Expectations)

Catching bad data early with reusable, AI-assisted checks

Tech stack: Python, Great Expectations, AI-assisted check generation

Problem statement

Bad data was found late, after it reached reports, and writing checks by hand for every table took too much effort.

How it works

```
Table / dataset
> AI helps generate expectations (rules)
> Great Expectations runs checks (nulls, types, ranges, uniqueness)
> Results logged to control / audit tables
> Pass: data promoted. Fail: blocked and alerted
```

Key design points

- Checks run before data is promoted to the next layer.
- AI speeds up writing and maintaining expectations, with human review.
- Results are logged, giving an audit trail.

My role

[fill in which project it was built for]

Challenges and solutions

Challenge	How it was solved
Too many tables to write checks for by hand	AI-assisted generation with human review
Noisy false alarms	Tuned rules; critical rules stop the pipeline, softer ones warn

Impact

Issues caught early, less manual checking effort.

Spoken version

I built a validation framework using Great Expectations, with AI helping generate and maintain the checks. It validates things like nulls, types and uniqueness before data is promoted, and logs results for audit. This catches bad data early instead of after it reaches a report.

7. Veeva Document Migration (Dynatech Systems)

SharePoint to Veeva using Databricks, OCR and APIs

Tech stack: Databricks (Python), OCR, Azure Blob Storage, REST APIs

Problem statement

Documents in SharePoint lacked structured metadata and had to move into Veeva accurately and at scale.

How it works

```
SharePoint documents
> OCR-based extraction
> Metadata mapping, stored in Azure Blob
> Spec ID auto-generation
> Staging upload to Veeva via API
> Final posting to Veeva via API
```

Key design points

- OCR extracts metadata hidden inside documents.
- Spec IDs generated automatically for consistency.
- A staging step before final posting makes loading safer.

My role

Built the end-to-end migration workflow.

Challenges and solutions

Challenge	How it was solved
Metadata hidden inside documents	OCR-based extraction
Consistent unique IDs	Automated Spec ID generation
Safe loading	Staging upload before the final post

Impact

A manual migration became an automated, repeatable pipeline.

Spoken version

We migrated documents from SharePoint to Veeva. I built OCR-based extraction and mapping in Databricks, stored metadata in Azure Blob, generated Spec IDs automatically, and uploaded through staging to final posting via APIs.

8. PaceLogic Healthcare / MPH Migration and ETL (Microsoft Fabric)

Excel and multi-source data to structured tables and Dataverse

Tech stack: Microsoft Fabric, Python/PySpark notebooks, SQL CTEs, Copy Activity, Dataverse, Great Expectations

Problem statement

Business data lived in Excel files and multiple sources, and it needed to move into structured tables and Dataverse reliably.

How it works

```
Excel and other sources
> Structured tables (Fabric Lakehouse) via PySpark notebooks
> Type casting, mapping, SQL CTE logic, module by module
> Copy Activity pipelines: Lakehouse -> Dataverse
> Scheduled pipelines and audit logging
> Data quality: Great Expectations, control tables, null handling
```

Key design points

- Module-wise migration and verification.
- Automated scheduling and audit logging.
- Quality checks with Great Expectations, control tables and null handling.

My role

Built the ETL pipelines and quality checks.

Challenges and solutions

Challenge	How it was solved
Messy Excel data	Type casting and mapping
Verifying each module	Control tables and CTE-based checks
Traceability	Audit logging

Impact

Reliable, verifiable, scheduled migration into Dataverse.

Spoken version

I migrated Excel data into structured tables in Microsoft Fabric using PySpark notebooks, then used Copy Activity pipelines to move it from Lakehouse to Dataverse. I automated scheduling and audit logging and applied Great Expectations, control tables and null handling for quality.

9. Azure Function App, Belc Ese Base (Thinkbiz Technology)

Timer-triggered automation for hourly and daily processing

Tech stack: Python, Azure Function App (timer trigger), Azure Blob Storage, SQL

Problem statement

Data processing ran manually every hour and every day.

How it works

```
Timer trigger (hourly / daily)
> Read binary .gz files from Blob Storage
> Backup before processing
> Truncate target tables (clean load)
> Load raw data into the database
> Execute stored procedures
```

Key design points

- Timer trigger fits the scheduled workload.
- Backup before processing allows safe reprocessing.
- Truncate before each trigger guarantees no duplicates.

My role

Developed the function app.

Challenges and solutions

Challenge	How it was solved
Safe reprocessing	Backup before processing
Duplicates	Truncate before each trigger
Binary compressed files	Handled .gz in Python

Impact

Hourly and daily processing fully automated.

Spoken version

I built an Azure Function App with a timer trigger to automate hourly and daily processing. It reads .gz files from Blob Storage, backs them up, loads raw data into the database after a truncate, and runs stored procedures.

10. Lumen Data Cleanse (Tech Mahindra)

Telecom revenue analysis to fix billing discrepancies

Tech stack: Snowflake, Matillion, SQL, Python, Power BI, AWS S3, EC2

Problem statement

Billing discrepancies in telecom revenue data were causing revenue leakage.

How it works

```
Source data
> Matillion ETL into Snowflake
> Validation framework flags discrepancies
> Power BI dashboards for stakeholders
```

Key design points

- Solution built from scratch with SQL and Matillion.
- Validation framework designed in Snowflake.
- Custom ETL built with cross-functional teams.

My role

Built the solution and validation framework.

Challenges and solutions

Challenge	How it was solved
Finding billing discrepancies	Validation rules in Snowflake
Making results usable	Power BI dashboards

Impact

Reduced revenue leakage losses by 15%.

Spoken version

At Tech Mahindra, I built a telecom revenue analysis solution from scratch using SQL and Matillion. I designed validation frameworks in Snowflake to find billing discrepancies, which reduced revenue leakage by 15%, and built Power BI dashboards for stakeholders.

11. Docker and Kubernetes POCs

Containerized and deployed workloads as proofs of concept

Tech stack: Docker, Kubernetes **[fill in what you containerized and where Kubernetes ran]**

Problem statement

Solutions that worked on one machine often behaved differently in another environment, and there was no easy way to package, deploy and scale them.

How it works

```
Application code
> Docker image (dependencies and config packaged)
> Kubernetes Deployment (replicas, restarts)
> Service for stable access
```

Key design points

- Docker keeps behavior consistent across environments.
- Kubernetes runs multiple replicas and restarts failed containers.
- Deployments are repeatable and portable.

My role

[fill in]

Challenges and solutions

Challenge	How it was solved
Environment differences	Packaged everything in Docker images
Keeping services running	Kubernetes Deployments with replicas

Impact

Repeatable, portable deployments; showed the approach can scale beyond one machine.

Spoken version

I've built proofs of concept using Docker and Kubernetes, containerizing workloads so they behave the same in every environment and using Kubernetes to deploy and manage them, with replicas and automatic restarts.

3. Follow-up Questions and Answers

Answers are written in first person so you can say them as they are. Highlighted items need your own detail.

Loan Analysis

Q. How did you measure the 40% improvement in accuracy?

I compared matched records against a manually verified sample, before and after adding fuzzy matching. Exact joins missed records with spelling or formatting differences, and fuzzy matching recovered them. **[confirm your actual method]**

Q. Why RapidFuzz?

It is fast, actively maintained and has a clean API. It scores string similarity from 0 to 100 and has scorers like token_sort_ratio for names with reordered words. It is also much faster than FuzzyWuzzy.

Q. How did you choose the matching threshold?

I tested a range of cutoffs on a verified sample and picked the one that balanced false matches against missed matches. Borderline scores are not auto-accepted. **[confirm]**

Q. How do you avoid false matches?

I normalize text first (lowercase, trim, remove punctuation and suffixes like Ltd), use a high cutoff, and keep low-confidence matches out of the final report.

Q. How did you handle performance on large data?

I avoided comparing every record against every other by narrowing candidates first. **[confirm what you actually did: blocking key, process.extractOne against a candidate list, UDF in Spark]**

Q. How does the checkpoint work?

Before the Monday run, the pipeline checks whether Friday's report was already generated. If so, Monday exits without reprocessing, avoiding duplicate work and compute cost. **[confirm where the status is stored]**

Q. What if a vendor API fails or returns incomplete data?

Each source is processed independently, so one failure does not corrupt the others. Failures trigger an email alert to stakeholders and the team, and the run can be retried. **[confirm retry behavior]**

Q. How is the report published to SharePoint?

Through an API call after the final report is generated, authenticated with a service credential rather than a personal login. **[confirm]**

Q. Why Delta Lake?

ACID transactions, versioning (time travel) and schema enforcement, which matter for financial reports.

Q. How do you handle schema changes from vendors?

Each source is cleansed in its own step, so mapping changes stay isolated. Schema checks catch unexpected columns early.

Loss Stats

Q. Why four staging tables and sixteen final tables?

The four staging tables mirror the template inputs, where data is validated. The sixteen final tables follow the data model of the old Access system, so downstream views and subroutines work the same. **[confirm]**

Q. How did you prove the Databricks output matched the old Access output?

We ran the same inputs through both systems and compared row counts, totals and key fields table by table. Differences were investigated before sign-off.

Q. How does the incremental load work?

Only new or changed data from each uploaded template is processed. Records are compared against existing data and merged. **[confirm if you used MERGE or a key and timestamp comparison]**

Q. What does ETL testing at each layer check?

Row counts, null checks, data types, duplicates, key integrity and business rules, before data is promoted from staging to final.

Q. Why ELT instead of ETL?

We load data first and transform inside Databricks, where compute scales, rather than transforming before loading.

Q. What do the subroutines do?

They are the business logic that lived in MS Access. We reimplemented them in Python, PySpark or SQL and run them between staging and final loads. **[confirm]**

Q. How did you split work among your three team members?

By layer or module, such as ingestion and validation, transformations, and testing, while I owned the data model, reviews and progress reporting. **[confirm]**

Q. How do you report progress to management and DevOps?

Regular status updates tracking tasks, blockers and delivery against plan, plus work items on DevOps. **[confirm cadence]**

Q. What was the hardest part?

Reproducing years of undocumented Access logic exactly. We documented each table, view and subroutine first, then validated outputs side by side.

Q. What happens if a template fails validation?

It is rejected at the validation or pre-staging step with a clear error, so bad data never reaches staging or final tables.

CI/CD and DevOps

Q. Walk me through your CI/CD pipeline.

Code is committed to Git on a branch, reviewed through a pull request, and automated checks run. After merge, the pipeline deploys to the next environment. **[confirm your tool (Azure DevOps or GitHub) and stages]**

Q. How do you deploy Databricks code?

Code lives in Git-linked repos and is promoted across environments through the pipeline instead of copying notebooks by hand. **[confirm]**

Q. Why CI/CD for data pipelines?

It makes releases repeatable, reduces manual errors, keeps history and allows rollback.

Q. How do you manage secrets and credentials?

Through a secret store (for example Databricks secret scopes or a key vault), never in code. **[confirm]**

Docker and Kubernetes

Q. What did you containerize?

[fill in, e.g. a FastAPI app or a PySpark job]

Q. Container vs VM?

A container shares the host OS kernel and packages only the app and its dependencies, so it is lightweight and starts fast. A VM includes a full guest OS, so it is heavier.

Q. What are a Pod, a Deployment and a Service?

A Pod is the smallest unit and runs one or more containers. A Deployment manages replicas and rolling updates of Pods. A Service gives Pods a stable network address and load balancing.

Q. How does Kubernetes handle a failed container?

It restarts it automatically, and a Deployment keeps the desired number of replicas running.

Q. Where did you run Kubernetes?

[fill in, e.g. Minikube, kind, AKS or EKS]

Q. Image vs container?

An image is the packaged template. A container is a running instance of it.

Q. Why Docker?

Consistency. The same image behaves the same on any machine, removing 'works on my machine' problems.

Internal AI Agent for DevOps

Q. Why keep a human in the loop?

DevOps items drive planning, so wrong changes are costly. The agent drafts and a person approves, which keeps speed and control.

Q. How does the agent get its input?

From a requirement or update request given by the user. **[confirm]**

Q. How do you stop wrong updates?

Approval before saving, and a clear preview of what will change.

Q. Which tool powers it?

Our internal AI tool, Spark AIR. **[confirm which components you can discuss]**

Q. How did you measure time saved?

By comparing the time to write and update items manually against reviewing the agent's drafts. **[add your real number]**

Q. What are the limits of the agent?

It can misread vague requirements, so quality depends on clear input and human review.

VoxShield AI (Hackathon, 2nd Place)

Q. What problem does VoxShield AI solve?

Voice-clone fraud. Most detection tools only say yes or no, but real scams splice AI-generated segments into a real voice at critical moments. VoxShield shows not just IF a voice is AI, but WHERE in the audio the AI segments appear.

Q. Walk me through the five layers.

Layer 1 decodes and validates the audio. Layer 2 runs the ML model and scores each chunk. Layer 3 does forensic analysis: chunk statistics, splice detection and pattern matching. Layer 4 turns findings into human-readable explanations. Layer 5 combines everything into a calibrated risk score and level.

Q. How does splice detection work?

Each audio chunk gets an AI probability. A sharp jump between adjacent chunks, over 35% in our design, is flagged as a splice point. The timeline then shows where AI audio was inserted. **[confirm the chunk length]**

Q. Which models are in the ensemble?

[confirm. The deck mentions a wav2vec2 deep learning model, Gemini and HuggingFace models, and acoustic features such as jitter, shimmer and pitch.]

Q. Why accuracy over latency?

Each analysis takes 15 to 30 seconds, because banks need forensic-grade evidence. Quick shortcuts give unreliable yes/no answers that cannot be used in fraud investigations.

Q. How did you handle mixed-language audio?

Hindi and English code-switching can cause variance in chunk probabilities, so we flag it and adjust the confidence.

Q. How do you avoid false positives on robotic human voices?

Call center agents and nervous speakers can sound robotic, so uniformity analysis separates natural variance from the consistency of synthetic speech.

Q. What is the AI bookend pattern?

Scammers wrap an AI-generated intro and outro around a real human middle section. Pattern detection catches that specific tactic.

Q. Where does the system struggle?

Audio under 5 seconds (not enough data, so we flag low confidence), heavy noise (about 78-82% accuracy), and brand-new TTS models (about 85-88%). We mitigate with the ensemble and a wav2vec2 model that generalizes better than fixed rules, and plan continuous monitoring for new AI models.

Q. How did you measure accuracy?

[confirm your test data and method, for example a labeled set of real and AI-generated clips per language]

Q. What is the business model?

API-as-a-service with a free tier (100 checks per month), a startup tier ($99 per month, 5,000 checks) and custom enterprise pricing, at roughly $0.007 per request. Targets are banks and NBFCs, fintech and UPI, call centers and insurance.

Q. What would you improve with more time?

Continuous monitoring to detect new AI voice models within days of release, regular updates to feature thresholds, and better handling of very short and very noisy audio.

Q. What was your role and team size?

[fill in] The team, ANALYTICSWITHANAND, had four members.

RegimeTrader Pro

Q. What is a Hidden Markov Model?

A model where the market has hidden states (regimes) we cannot see directly, but we observe returns and volatility that depend on the state. The model learns the states and the probabilities of moving between them.

Q. Why HMM?

Markets behave differently in different conditions. HMMs detect those regimes without needing labels.

Q. Why five regimes?

Bull, Bear, Neutral, Crash and Euphoria cover the main market behaviors, and each leads to a different trading action. **[confirm how you chose five]**

Q. How did you avoid overfitting?

Ten years of data across twelve stocks, plus combining the regime with news sentiment and risk controls instead of trusting one signal. **[confirm if you did out-of-sample testing]**

Q. How is sentiment scored?

News and posts from Yahoo Finance, Google News and Reddit are scored, then combined into one signal. **[confirm your method]**

Q. Did you backtest it?

[answer honestly; it runs on Alpaca paper trading, so be clear it is not real money]

Q. What risk controls did you build?

Circuit breakers, a kill switch and dynamic position sizing.

Q. Why paper trading?

It tests the full execution flow without risking real money.

Data Validation Framework

Q. What is Great Expectations?

A Python library where you define expectations (rules) about data, such as no nulls, valid ranges and unique keys, then run them and get a pass or fail report.

Q. How did AI help?

It helped generate and maintain expectations from schemas and sample data, with a human reviewing them. **[confirm]**

Q. What happens when a check fails?

The data is blocked from promotion, the failure is logged, and an alert is sent.

Q. How do you avoid noisy false alarms?

Tune thresholds, and separate critical rules that stop the pipeline from warning-level ones.

Q. Why not just write SQL checks?

Great Expectations is reusable, standardized, documents the rules and produces reports.

Veeva, Fabric, Azure Function, Lumen

Q. Veeva: which OCR tool, and how did you handle poor scans?

[fill in the tool]. For poor scans, I validated extracted fields and flagged low-confidence documents for review.

Q. Veeva: how did you verify the migration?

By comparing document counts and metadata between source and target, and using a staging upload before final posting.

Q. Fabric: why Copy Activity?

It moves data between Lakehouse and Dataverse reliably, with built-in scheduling and monitoring.

Q. Fabric: what are control tables?

Tables that track what was loaded and its status, which helps with audit, reruns and verification.

Q. Azure Function: why a timer trigger?

The processing was scheduled (hourly and daily), so a timer fits better than an event trigger.

Q. Azure Function: why truncate before each load?

To guarantee a clean load with no duplicates, with a backup taken before processing.

Q. Lumen: how did you quantify the 15%?

By comparing revenue discrepancies identified and recovered against the earlier leakage baseline. **[confirm]**

Q. Lumen: why Matillion with Snowflake?

Matillion pushes transformations down into Snowflake, which scales well and keeps ETL simple.

General Questions

Q. What is your biggest achievement?

Improving report accuracy by 40% on Loan Analysis and automating a process that used to be fully manual.

Q. Describe a challenge and how you handled it.

Loss Stats: replicating undocumented Access logic exactly was hard, so I documented every table, view and subroutine first, then validated outputs side by side at each layer.

Q. How do you lead a team?

Clear ownership, regular progress updates, code reviews, and unblocking people quickly.

Q. Why are you looking for a change?

I'm looking for bigger scope in AI-ready data platforms, where I can grow as both an engineer and a leader.

4. Checklist Before the Interview

- **Measure of 40%:** know exactly how accuracy was measured (verified sample, before and after).
- **Incremental load:** be able to explain how new or changed records are detected (MERGE, key and timestamp).
- **CI/CD:** know your tool (Azure DevOps or GitHub) and the stages from commit to deploy.
- **Docker and Kubernetes:** know what you containerized and where Kubernetes ran.
- **AI agent:** have one real number for time saved.
- **VoxShield AI:** know your role in the team of four, which models were in the ensemble, and how the accuracy figures were measured.
- **RegimeTrader Pro:** be honest that it runs on paper trading; know how you chose five regimes.
- **Numbers you quote:** 40% accuracy, 15% leakage reduction, 4 staging and 16 final tables, team of 3. Only quote what you can explain.
- **If you do not know a detail:** say what you do know and how you would find out. It sounds more credible than guessing.

Closing lines

- **Strength:** I combine solid data engineering with hands-on AI work, and I take ownership from design to delivery.
- **Why a move:** I'm looking for bigger scope in AI-ready data platforms, where I can grow both as an engineer and a leader.

Good luck, Prachi.